

MEMBER 1

Infrastructure & Orchestration

Environment · HDFS · PostgreSQL · Sqoop · ClickHouse · Airflow

Name: _____

The shared table contract, metric definitions, risks and timeline are in **00_Team_Overview**. Read it first. This brief only covers your role.

YOUR FIRST 3 HOURS

- Hour 0: run the 5-minute VM health check** (commands below). Check daemons, disk ≥ 40 GB free, RAM ≥ 8 GB, Python version and `yum`.
- Decide VM or Docker by hour 2.** If anything fails, switch to Docker Compose within the first 2 hours. Confirm `docker-compose.yml` exists in the project root. Create it only if the VM health check fails.
- Hour 1: agree the contract with the team.** The 8 tables and the metric definitions are in `00_Team_Overview`.
- Start ClickHouse.** Create the database `flight_analytics` and the dummy table `connectivity_test` (both are in the DDL below). Make port 8123 reachable from Windows.
- Tell Member 4 as soon as ClickHouse is reachable.** Member 4 must connect Power BI to your dummy table by hour 3.

Your deliverables

- VM health check result, or the Docker stack
- HDFS layout, ingestion, lookup tables (raw CSVs with `hdfs dfs -put`; carrier / airport lookups with Sqoop)
- ClickHouse database + all 8 tables (DDL below)
- `flight_pipeline_dag.py` (BashOperator tasks), with one green run
- Run-times and storage-size log; Airflow green tree-view screenshot; master `README.md`
- Content for slides 3 (Dataset), 4 (Architecture) and 5 (Storage & relational ingestion)

Definition of done

- ☐ Health check done. VM or Docker decided by hour 2. Disk, RAM and Python checked.
- ☐ ClickHouse running. `flight_analytics` and the dummy table exist. Port 8123 reachable from Windows. Member 4 is connected.
- ☐ The 1-month sample (`2024_1.csv`) is in HDFS at `/project/flights/sample`.
- ☐ Carrier lookup table created in PostgreSQL (or MySQL).
- ☐ Sqoop import PostgreSQL $\rightarrow$ HDFS done within 2 hours, **or** the fallback is used and documented.
- ☐ All 8 ClickHouse tables created from the DDL.
- ☐ Airflow runs in a Python ≥ 3.9 venv (or the official `apache/airflow` image in `standalone` mode).
- ☐ `flight_pipeline_dag.py` has the tasks `check_data` $\rightarrow$ `csv_to_parquet` $\rightarrow$ `spark_aggregates` $\rightarrow$ `load_clickhouse` $\rightarrow$ `train_model` as BashOperators.
- ☐ All 48 CSVs are in `/project/flights/raw/`.
- ☐ Run times and storage sizes logged.
- ☐ DAG triggered. Screenshot of the green tree view taken.
- ☐ Master `README.md` assembled.
- ☐ Content ready for slides 3, 4, 5. Slide 5 says honestly if Sqoop fell back.

Your schedule, hour by hour

Hours count from kickoff. Text is taken from the plan's schedule tables.

When	What you do (from the plan)
Day 1 — foundation, parallel work on the 1-month sample (hours 0–24)	
Hours 0–3	Health check (daemons, disk ≥ 40 GB, RAM, Python version, <code>yum</code>). Decide VM vs. Docker by hour 2. Start ClickHouse , create <code>flight_analytics</code> and one dummy table; make port 8123 reachable from Windows.

When	What you do (from the plan)
Hours 3–8	Load the 1-month sample to HDFS (<code>/project/flights/sample</code>). Create PostgreSQL (or MySQL) carrier lookup table.
Hours 8–14	Sqoop import (2-hour time-box) PostgreSQL → HDFS; fall back as described in the Sqoop warnings below if needed. Then set up Airflow in a Python ≥ 3.9 venv.
Hours 14–20	Create all 8 ClickHouse tables from the DDL (see the DDL below). Start the Airflow DAG skeleton.
Hours 20–24	Airflow draft: <code>check_data</code> → <code>csv_to_parquet</code> → <code>spark_aggregates</code> → <code>load_clickhouse</code> → <code>train_model</code> as <code>BashOperators</code> .
Day 2 — full 48-month run, polish, defense readiness (hours 24–48)	
Hours 24–30	Bulk-load the 48 CSVs to <code>/project/flights/raw/</code> .
Hours 30–36	Log run times and storage sizes.
Hours 36–42	Finalize and trigger the Airflow DAG; screenshot the green tree view. (If the full run is too long, trigger it on the Parquet-based stages.)
Hours 42–48	Assemble master <code>README.md</code> .

Handoffs: who sends what, by when

You receive from	What	By when
All members	The agreed contract: 8 tables + metric definitions	Hour 1
Member 4	Result of the Power BI ↔ ClickHouse test (DirectQuery works, or Import fallback)	By hour 3 †
Members 2, 3	Spark scripts and the training script, to wrap in the DAG tasks	Hours 20–24 †

You send to	What	By when
Member 4	ClickHouse running, <code>flight_analytics</code> + dummy table, port 8123 reachable from Windows	Hours 0–3
Member 2	1-month sample in HDFS: <code>/project/flights/sample</code>	Hours 3–8 †
Members 2, 3	All 8 ClickHouse tables created from the DDL (loads need them)	Hours 14–20 †
Member 2	48 CSVs in <code>/project/flights/raw/</code>	Hours 24–30 †
Member 4	Run times and storage sizes; green Airflow tree-view screenshot	Hours 30–42 †
Member 4	Content for slides 3–5	Before slides 1–6 are built (hours 24–30) †

† Not written as a handoff in the plan, but the schedule needs it. Agree the exact hour with the other member.

HDFS layout

Path	What goes there
<code>/project/flights/raw/</code>	Raw CSV: 48 monthly files
<code>/project/flights/sample</code>	1-month sample (<code>2024_1.csv</code>)
<code>/project/flights/parquet/</code>	Cleaned Parquet (~35 columns), partitioned by <code>Year</code>
<code>/project/reference/</code>	Carrier / airport lookup tables

Warnings for your role

- **Sqoop: 2-hour time-box.** Sqoop is a retired Apache project. Version 1.4.7 often fails on Hadoop 3 (missing JDBC driver / commons-lang jars in `$SQOOP_HOME/lib`, `HADOOP_MAPRED_HOME`). Stop after 2 hours. If Sqoop already works on the NTI VM from the course, use that setup.

- **Sqoop fallback.** Load the lookup CSV with `hdfs dfs -put`. Keep the Sqoop command and the PostgreSQL schema in the README as the designed path. Slide 5 must say this honestly. Do not claim Sqoop ran.
- **Sqoop is only for relational lookup data** (carrier / airport tables). Raw CSVs go in with `hdfs dfs -put`.
- **CentOS is end-of-life.** Default `yum` repos may fail (they need `vault.centos.org`). CentOS 7 system Python is 3.6. Do not use it for Airflow.
- **Airflow needs Python ≥ 3.9.** Use a Python ≥ 3.9 venv (not system Python) or the official `apache/airflow` image in `standalone` mode. Recent Airflow 2.x needs Python ≥ 3.8.
- **Disk and RAM.** Disk ≥ 40 GB free (raw 12.5 GB + Parquet + HDFS + images). RAM ≥ 8 GB. On a single node set `dfs.replication = 1`.
- **The Docker stack must contain namenode, datanode, spark-master, spark-worker, clickhouse and postgres.** Sqoop is VM-only (no official Docker image).
- **Do not copy the 12.5 GB through a bind mount from the Windows drive.** It is slow on WSL2. Copy it into the WSL filesystem or into HDFS first.
- **Power BI ↔ ClickHouse.** Member 4 tests it in hours 0–3. If it fails, check port 8123 and the VM network with Member 4. Fallback: Import mode.
- **Parquet, not raw CSV.** After the first read, all jobs read Parquet (~1.5–2 GB). Raw CSV runs take about 30 minutes.
- **Airflow polish is cut second** if we fall behind. Keep a simple DAG of `BashOperators` with one green run. If the full run is too long, trigger the Parquet-based stages.

Commands, DDL and code

Path A: VM health check (bash)

```
# 1. Resources and Python (CentOS end-of-life / old Python checks)
df -h ; free -g ; python3 --version ; java -version

# 2. Start core Hadoop daemons
start-dfs.sh && start-yarn.sh
jps          # MUST show: NameNode, DataNode, ResourceManager, NodeManager

# 3. HDFS write test
hdfs dfs -mkdir -p /project/flights/raw /project/flights/parquet /project/reference
hdfs dfs -ls /project/flights

# 4. Spark readiness
spark-shell --version

# 5. Can we install anything? (fails on EOL CentOS repos unless vault is configured)
yum --version && yum makecache 2>&1 | tail -3
```

Path B: Docker, if the VM fails (PowerShell)

```
# Needs namenode, datanode, spark-master, spark-worker, clickhouse, postgres
docker compose up -d
docker ps
```

ClickHouse DDL: all 8 tables + connectivity test (SQL)

```
CREATE DATABASE IF NOT EXISTS flight_analytics;
USE flight_analytics;

CREATE TABLE IF NOT EXISTS agg_airline_performance (
    Year UInt16, Reporting_Airline String, Airline_Name String,
    Total_Flights UInt32, OnTime_Flights UInt32, Delayed_Flights UInt32,
    Delay_Rate_Pct Float32, Avg_Dep_Delay_Min Float32, Avg_Arr_Delay_Min Float32,
    Cancelled_Flights UInt32, Cancellation_Rate_Pct Float32
) ENGINE = MergeTree() ORDER BY (Year, Reporting_Airline);

CREATE TABLE IF NOT EXISTS agg_airport_performance (
    Airport_Code String, Airport_City String, Airport_State String,
    Total_Departures UInt32, Total_Arrivals UInt32,
    Dep_Delay_Rate_Pct Float32, Arr_Delay_Rate_Pct Float32,
    Avg_Dep_Delay_Min Float32, Cancelled_Departures UInt32
) ENGINE = MergeTree() ORDER BY Airport_Code;

CREATE TABLE IF NOT EXISTS agg_delay_causes_monthly (
    Year UInt16, Month UInt8, Total_Delay_Minutes UInt64,
    Carrier_Delay_Min UInt64, Weather_Delay_Min UInt64, NAS_Delay_Min UInt64,
```

```
Security_Delay_Min UInt64, Late_Aircraft_Delay_Min UInt64,
Carrier_Pct Float32, Weather_Pct Float32, NAS_Pct Float32,
Security_Pct Float32, Late_Aircraft_Pct Float32
) ENGINE = MergeTree() ORDER BY (Year, Month);

CREATE TABLE IF NOT EXISTS agg_hourly_delays (
    Dep_Hour UInt8, Total_Flights UInt32, Delayed_Flights UInt32,
    Delay_Probability_Pct Float32, Avg_Delay_Minutes Float32
) ENGINE = MergeTree() ORDER BY Dep_Hour;

CREATE TABLE IF NOT EXISTS agg_route_traffic (
    Origin String, Dest String, Route_Name String, Flight_Count UInt32,
    Avg_AirTime_Min Float32, Distance_Miles UInt32, Route_Delay_Rate_Pct Float32
) ENGINE = MergeTree() ORDER BY (Origin, Dest);

CREATE TABLE IF NOT EXISTS agg_calendar_delays (
    Year UInt16, Month UInt8, Day_Of_Week UInt8,
    Total_Flights UInt32, Delayed_Flights UInt32, Cancelled_Flights UInt32,
    Delay_Rate_Pct Float32, Avg_Arr_Delay_Min Float32
) ENGINE = MergeTree() ORDER BY (Year, Month, Day_Of_Week);

CREATE TABLE IF NOT EXISTS agg_cancellation_reasons (
    Year UInt16, Reporting_Airline String, Cancellation_Code String,
    Cancellation_Reason String, Cancelled_Flights UInt32
) ENGINE = MergeTree() ORDER BY (Year, Reporting_Airline, Cancellation_Code);

CREATE TABLE IF NOT EXISTS ml_delay_predictions (
    FlightDate Date, Reporting_Airline String, Origin String, Dest String,
    Dep_Hour UInt8, Actual_Delayed UInt8, Predicted_Delayed UInt8,
    Delay_Probability Float32, Risk_Category String
) ENGINE = MergeTree() ORDER BY (FlightDate, Reporting_Airline);

-- Hours 0-3 connectivity test table (drop later)
CREATE TABLE IF NOT EXISTS connectivity_test (id UInt8, note String) ENGINE = MergeTree() ORDER BY id;
INSERT INTO connectivity_test VALUES (1, 'power bi test');
```